Chromatica: Spotify Data Visualization System

Serjo Barron

1 Project Overview

Chromatica is a full-stack web application built with Next.js that integrates with the Spotify API to transform user listening data into a dynamic visual experience. The system retrieves personalized listening metrics, such as top tracks, artists, and time ranges, and converts them into a reactive UI driven by color palettes extracted from album artwork.

The goal was to build a modern full-stack system that ties together authentication, external APIs, and persistent data storage into a cohesive user-facing experience. The project was also influenced by existing Spotify-based visualization tools, with a focus on building a current implementation using modern web development tooling.

2 System Architecture

The system is implemented using a full-stack Next.js architecture deployed on Vercel, consisting of:

- **Frontend:** React (Next.js) for UI rendering and state-driven visualization
- **Backend:** Next.js API routes handling authentication, Spotify API communication, and server-side logic
- **Database:** PostgreSQL hosted on Neon, accessed via Prisma ORM
- **External API:** Spotify Web API using OAuth 2.0 authentication
- **Deployment:** Vercel serverless platform for hosting both frontend and API routes

Users authenticate via Spotify OAuth, receive access tokens, and have their profile and session data persisted in PostgreSQL. API routes act as the bridge between the frontend and Spotify's services.

3 Core Features

The application is composed of several interconnected systems:

- **Authentication System:** OAuth 2.0 integration with Spotify, including token generation, refresh handling, and session persistence

- **Data Aggregation Layer:** Retrieval of user-specific listening data such as top artists, tracks, and time-range-based statistics
- **Dynamic Theming Engine:** Extraction of dominant color palettes from album artwork using node-vibrant
- **Reactive UI System:** Frontend components that dynamically update based on fetched and processed Spotify data
- **Persistence Layer:** PostgreSQL schema designed to store user session data and authentication tokens

4 Key Engineering Decisions

Next.js was chosen as the core framework because it unifies frontend rendering and backend API routes in a single codebase, letting UI components and server-side logic interact closely without needing a separate backend service. API route boundaries were still maintained to preserve separation of concerns while keeping deployment simple.

Prisma was used as the ORM layer for type-safe database interactions and straightforward schema management against a PostgreSQL database hosted on Neon. All Spotify API communication was handled server-side to keep authentication tokens secure and off the client, which also centralized rate limit handling and response formatting. React state management on the frontend handled updating UI components as data came in.

5 Technical Challenges

The most significant challenges involved integrating external APIs with a reactive frontend while keeping data flow consistent across asynchronous operations. Implementing the Spotify OAuth 2.0 flow required careful handling of access tokens, refresh tokens, and session persistence, with coordination between client-side state and server-side API routes to keep authentication stable across sessions.

Working with PostgreSQL and Prisma also had a learning curve around designing a relational schema suited for authentication persistence without overcomplicating the data model. Getting a local development environment that matched production required stable database connectivity with Neon before deploying, which was necessary to catch any discrepancies between local and deployed behavior early.

6 Tradeoffs and Constraints

Using Next.js API routes instead of a separate backend kept infrastructure simple and development fast, but introduced limitations around backend scalability and support for long-running processes. These constraints shaped how server-side logic and API interactions

were structured.

Deploying on Vercel introduced serverless constraints like stateless function execution, which affected how persistent operations and repeated API calls were handled. Real-time data fetching was prioritized over caching to keep things simple and ensure up-to-date Spotify data. The database schema was kept intentionally minimal, focused on authentication and session management rather than large-scale data storage.

7 System Limitations

Chromatica was designed as a single-user or small-scale demonstration application, largely due to Spotify API access constraints encountered during development. At the time of implementation, user onboarding required manual validation, limiting open public registration. External API policies also restrict certain developer access levels, which reinforces its role as a demonstrative project rather than a production multi-tenant system.

8 Future Improvements

Several improvements could be made to enhance scalability and system robustness:

- Introducing caching layers such as Redis or edge-based caching to reduce redundant API calls
- Improving authentication persistence and token lifecycle management
- Offloading computational tasks such as color extraction to background processing systems
- Expanding system design to support multi-user scalability if API constraints are relaxed

9 Conclusion

Chromatica covers the full lifecycle of building a web application: external API integration, OAuth authentication, persistent data storage, and a reactive frontend. The project surfaced practical challenges around third-party API constraints, authentication management, and serverless deployment that aren't always obvious until you're working through them.

10 Links

- Live Demo
- GitHub Repository
- Figma Prototype